

TECHNICAL ARCHITECTURE BLUEPRINT

Community & Experiences Discovery Platform

Senior Software Architect Review · June 2026

MVP Target 3 Months	Growth 100K Users	Scale 1M+ Users	Immersive Phase 5
---------------------	-------------------	-----------------	-------------------

1. Recommended Technology Stack

Every choice below is optimized for fast MVP execution, startup cost efficiency, and a clean upgrade path to enterprise-grade scale. No over-engineering in Phase 1.

Category	Recommended	Why	Alternatives
Frontend	Next.js 15 (App Router)	SSR + SSG, SEO, API routes, full-stack React. Zero rewrite for web.	Remix, Nuxt (Vue), SvelteKit
Mobile	React Native + Expo	Share ~60–70% logic/UI with web via NX monorepo. Fastest cross-platform.	Flutter, Kotlin/Swift
Backend	FastAPI (Python)	Async, type-safe, auto-docs, native ML/AI ecosystem, familiar stack.	NestJS (Node), Go/Gin, Django
Language	TypeScript + Python	TS for frontend/mobile; Python for backend + AI pipelines.	Go (later services)
Primary DB	PostgreSQL 16	ACID, JSONB, pg_trgm fuzzy search, pgvector, mature ecosystem.	MySQL, CockroachDB
Cache / Queue	Redis (Upstash)	Session cache, pub/sub, queues via BullMQ. Serverless-friendly.	Memcached, NATS
Auth	Clerk	Email/social/SSO, MFA, RBAC, webhooks, mobile SDKs. Ships fast.	Supabase Auth, Auth0, custom JWT
File Storage	Cloudflare R2	Zero egress fees, S3-compatible, fast global CDN. Very cheap.	AWS S3, Supabase Storage
Search	Meilisearch	Sub-50ms, typo-tolerant, easy setup. Migrate to Elastic at 500K+ users.	Typesense, Algolia, Elasticsearch
Notifications	Firebase (FCM) + Resend	FCM for push (iOS/Android/Web). Resend for transactional email.	OneSignal, SendGrid, Expo Push
Payments	Stripe	Industry standard, Stripe Connect for marketplace/splits, great DX.	Paddle, Lemon Squeezy
AI / LLM	OpenAI API + pgvector	GPT-4o for assistant; ada-002 / text-	Anthropic, Cohere, local LLMs

		embedding-3 for embeddings + pgvector.	
3D / Immersive	Three.js + React Three Fiber	Declarative 3D in React. React Native Three Fiber for mobile.	Babylon.js, PlayCanvas, Unity WebGL
Analytics	PostHog (OSS)	Product analytics, session replay, feature flags, free self-host.	Mixpanel, Amplitude, June
Monitoring	Sentry + Grafana + Loki	Error tracking (Sentry), metrics (Grafana), structured logs (Loki).	Datadog, New Relic
CI/CD	GitHub Actions + Docker	Free for public, matrix builds, Compose for local parity.	GitLab CI, CircleCI
Hosting (MVP)	Railway	Deploy Postgres + Redis + FastAPI in minutes, auto-scale. \$5/mo start.	Render, Fly.io, Heroku
Hosting (Growth)	AWS (ECS + RDS + CloudFront)	Full control, reserved instances, global CDN, mature tooling.	GCP, Azure, Hetzner
IaC	Terraform + Pulumi	Terraform for AWS infra; Pulumi if team prefers TypeScript IaC.	CDK, Ansible

Key Justifications

- **Why FastAPI over Node.js/NestJS:** Python's AI/ML ecosystem (PyTorch, LangChain, scikit-learn) removes the boundary between backend and ML — no microservice split needed until Phase 4.
- **Why Next.js over plain React:** Built-in SSR means event pages and club profiles are SEO-indexed on Day 1 — critical for organic discovery growth.
- **Why Clerk over rolling your own auth:** Auth is a security risk center. Clerk ships magic links, OAuth, MFA, and session management in hours. Build your product, not an auth system.
- **Why Cloudflare R2:** AWS S3 charges egress fees per GB — at scale this adds up significantly. R2 is S3-compatible with zero egress cost, making it the default choice for a startup.
- **Why PostgreSQL + pgvector:** One database for relational data AND vector embeddings in MVP avoids operational overhead. You delay a dedicated vector DB until your embedding workload justifies it.

2. System Architecture

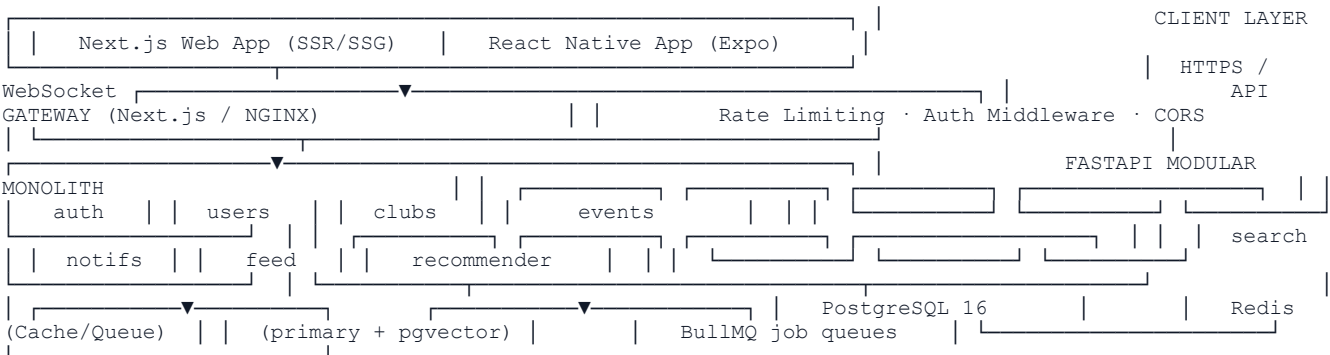
2.1 Architecture Evolution Strategy

Start with a well-structured Modular Monolith, not microservices. This is the most common mistake early startups make — splitting too early multiplies operational complexity by 10× before you have the traffic to justify it.

MVP Architecture — Modular Monolith (Months 0–6)

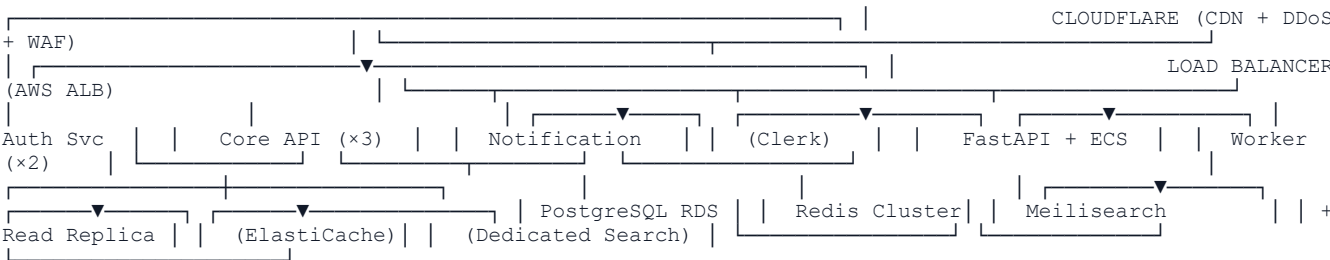
One FastAPI application, split into internal Python modules (packages) with clear domain boundaries. The structure mirrors what microservices would look like, making the future split mechanical rather than a rewrite.

Architecture Diagram (Text Form)

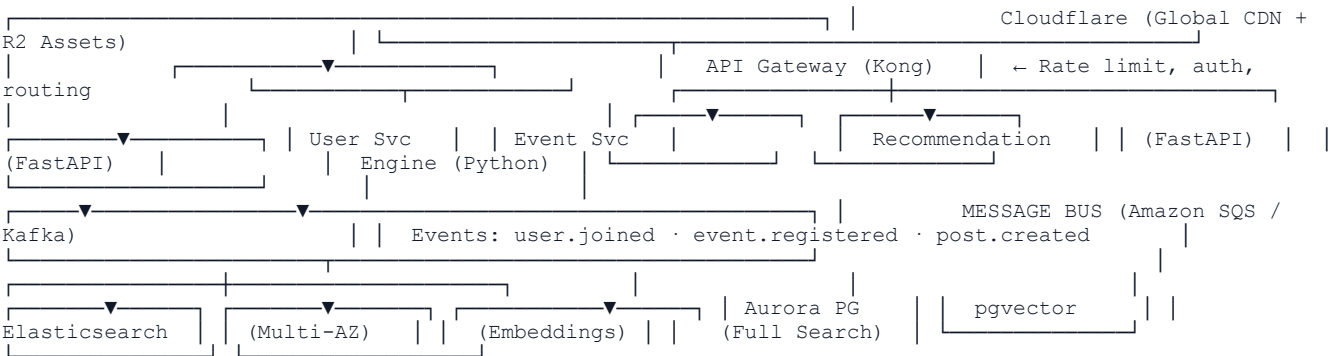


Growth Architecture (100K+ Users, Months 6–18)

Extract high-traffic services. Add read replicas. Introduce a dedicated search index.



Scale Architecture (1M+ Users)



2.2 API Design

API Strategy

API Type	Use Case	Benefit	When
REST (JSON)	Primary API for all CRUD operations, events, clubs, users.	Simple, cacheable, works everywhere	Phase 1 (MVP)
WebSocket	Real-time: notifications, live event capacity updates, chat.	Low-latency bidirectional	Phase 1
GraphQL	Mobile clients needing flexible queries, feed aggregation.	Reduces over-fetching on mobile	Phase 3
tRPC	Internal Next.js full-stack calls (server → client).	End-to-end type safety in monorepo	Phase 1 (web layer)

2.3 Caching Strategy

- **L1 — In-Memory:** LRU cache in FastAPI process. Club/event metadata hot paths. TTL: 60s.
- **L2 — Redis:** Session tokens, search results, recommendation outputs. TTL: 5–15 min.
- **L3 — CDN (Cloudflare):** Static assets, media files, public API responses (event listings). TTL: 1–24h.
- **DB Query Cache:** Materialized views for aggregated analytics (registration counts, event stats).

2.4 Background Jobs

- Email / push notification dispatch — BullMQ queue, Redis-backed
- Recommendation engine recomputation — nightly cron
- Embedding generation for new events/clubs — triggered on create/update
- Analytics aggregation — hourly rollup job
- Media processing (image resize, video thumbnail) — S3/R2 event trigger → Lambda/worker
- Waitlist promotion checks — event-triggered + periodic sweep

3. Mobile Strategy

3.1 Recommendation: React Native (Expo)

Given the frontend is Next.js (React), using React Native maximizes code reuse across web and mobile. This is the pragmatic startup choice.

Code Sharing Breakdown

Layer	Shared	Platform-Specific
Business Logic	100% shared (API calls, validation, state management, Zustand/Jotai stores)	Nothing
Types / Schemas	100% shared (Zod schemas, TypeScript interfaces)	Nothing
API Client	100% shared (tRPC client or Axios instance)	Nothing
UI Components	~40% via React Native Web / shared design tokens	Navigation, gestures, native modules
Styles	Design tokens + Tailwind config shared; RN uses StyleSheet	Platform rendering
State Management	100% shared (Zustand/Jotai)	Nothing
Auth	Clerk SDK: same hooks, platform-aware SDKs	Native biometric login

Monorepo Structure (NX / Turborepo)

```
apps/    web/           ← Next.js 15    mobile/       ← Expo (React Native) packages/    ui/
← Shared component primitives (React Native Web)  api-client/    ← tRPC / Axios API hooks
types/    ← Zod schemas + TypeScript interfaces  utils/           ← Date helpers,
formatters, validators  config/           ← ESLint, Tailwind, tsconfig
```

Why Not Flutter?

- **Separate codebase:** Flutter is Dart — your Next.js React code cannot be shared at all. You maintain two entirely different applications.
- **AI/ML ecosystem:** React Native integrates with the web's JS ML ecosystem (ONNX runtime, TensorFlow.js). Flutter AI support is thinner.
- **Team knowledge:** Assuming React expertise from web work. Flutter requires learning Dart from scratch.

Flutter is the right choice if native feel and animation performance are the primary differentiators — for this platform, code velocity and DX matter more at this stage.

4. Database Design

4.1 Database Selection

- **Primary:** PostgreSQL 16 — relational data, JSONB flexibility, ACID compliance
- **Vectors:** pgvector extension on same Postgres instance (MVP) → dedicated Qdrant at 1M+ users
- **Cache:** Redis — sessions, hot data, job queues
- **Search Index:** pg_trgm (MVP) → Meilisearch (Growth) → Elasticsearch (Scale)
- **Media:** Cloudflare R2 (object storage) — not the DB

4.2 Initial Schema

```
-- Users CREATE TABLE users ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), clerk_id TEXT UNIQUE NOT NULL, username TEXT UNIQUE NOT NULL, email TEXT UNIQUE NOT NULL, full_name TEXT, avatar_url TEXT, bio TEXT, interests TEXT[], -- array of tags embedding VECTOR(1536), -- pgvector for recommendations created_at TIMESTAMPTZ DEFAULT NOW(), updated_at TIMESTAMPTZ DEFAULT NOW() ); -- Clubs / Communities CREATE TABLE clubs ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), slug TEXT UNIQUE NOT NULL, name TEXT NOT NULL, description TEXT, category TEXT, tags TEXT[], cover_url TEXT, logo_url TEXT, location TEXT, is_private BOOLEAN DEFAULT FALSE, capacity INT, embedding VECTOR(1536), owner_id UUID REFERENCES users(id), created_at TIMESTAMPTZ DEFAULT NOW() ); -- Events CREATE TABLE events ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), club_id UUID REFERENCES clubs(id) ON DELETE CASCADE, title TEXT NOT NULL, description TEXT, event_type TEXT, location TEXT, lat DECIMAL(9,6), lng DECIMAL(9,6), starts_at TIMESTAMPTZ NOT NULL, ends_at TIMESTAMPTZ, capacity INT, registered INT DEFAULT 0, status TEXT DEFAULT 'draft', -- draft|published|cancelled requires_approval BOOLEAN DEFAULT FALSE, embedding VECTOR(1536), created_at TIMESTAMPTZ DEFAULT NOW() ); -- Memberships CREATE TABLE club_members ( user_id UUID REFERENCES users(id), club_id UUID REFERENCES clubs(id), role TEXT DEFAULT 'member', -- member|moderator|admin status TEXT DEFAULT 'active', -- pending|active|banned joined_at TIMESTAMPTZ DEFAULT NOW(), PRIMARY KEY (user_id, club_id) ); -- Event Registrations CREATE TABLE registrations ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), user_id UUID REFERENCES users(id), event_id UUID REFERENCES events(id), status TEXT DEFAULT 'confirmed', -- confirmed|waitlisted|cancelled waitlist_pos INT, registered_at TIMESTAMPTZ DEFAULT NOW(), UNIQUE(user_id, event_id) ); -- Notifications CREATE TABLE notifications ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), user_id UUID REFERENCES users(id), type TEXT NOT NULL, title TEXT, body TEXT, data JSONB, read BOOLEAN DEFAULT FALSE, created_at TIMESTAMPTZ DEFAULT NOW() ); -- Posts (Phase 3 social layer) CREATE TABLE posts ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), author_id UUID REFERENCES users(id), club_id UUID REFERENCES clubs(id), event_id UUID REFERENCES events(id), content TEXT, media_urls TEXT[], likes INT DEFAULT 0, created_at TIMESTAMPTZ DEFAULT NOW() );
```

4.3 Entity Relationships

users —< club_members >— clubs users —< registrations >— events clubs —< events users >—< posts >— clubs users —< notifications events >— clubs (belongs_to) posts —< comments (Phase 3) posts —< likes (Phase 3)

4.4 Indexing Strategy

Index	Table.Column	Purpose
B-Tree	users.email, users.clerk_id	Fast auth lookup
B-Tree	events.club_id, events.starts_at	Club event listings, upcoming filter
B-Tree	registrations.(user_id, event_id)	Unique + fast lookup
GIN	clubs.tags, events.tags (array)	Tag-based filtering
GIN	clubs.name (pg_trgm)	Fuzzy text search

GIN	events.description (to_tsvector)	Full-text search
IVFFlat	users.embedding, events.embedding	pgvector ANN similarity search
BRIN	events.starts_at	Time-range scans (large tables)
Partial	events WHERE status='published'	Active events only — avoids full scan

4.5 Scaling Approaches

- **Phase 1:** Single Postgres instance on Railway. Daily pg_dump backups to R2.
- **Phase 2:** Add a read replica. Route analytics/reporting queries to replica.
- **Phase 3:** Partition events table by starts_at (range). Archive past events.
- **Phase 4:** Move embeddings to Qdrant. Use connection pooling with PgBouncer.
- **Phase 5:** Citus extension or Aurora (horizontal sharding) if writes exceed 50K/s.

5. AI Architecture

5.1 Implementation Phases

Phase	Feature	Tech	When
1 — MVP	Content-based recommendations	pgvector cosine similarity	Month 1–3
1 — MVP	Semantic search (clubs/events)	OpenAI embeddings + pgvector	Month 2–3
2 — Growth	Collaborative filtering	Matrix factorization (implicit)	Month 4–6
2 — Growth	AI Event Planning Assistant	GPT-4o + RAG over event catalog	Month 5–7
3 — Social	Smart content moderation	OpenAI Moderation API	Month 7–9
4 — AI	Personalized feed ranking	LightGBM or two-tower model	Month 9–12
4 — AI	Smart user-club matching	Fine-tuned embedding model	Month 10–14
5 — Immersive	AI trip planning (generative)	GPT-4o + function calling + Maps API	Month 14+

5.2 Recommendation System Architecture

USER ONBOARDING | ▼ Interest Extraction → generate user embedding (text-embedding-3-small) | store in users.embedding (pgvector) |
▼ RECOMMENDATION PIPELINE | — Content-Based: cosine_similarity(user.embedding, event.embedding) | — Collaborative: users who joined similar clubs also joined... | — Popularity: trending events in user's city (last 7 days) | — Hybrid Ranker: weighted blend → re-ranked list | ▼ CACHE in Redis (TTL: 30 min per user) | ▼
/api/recommendations endpoint → client

5.3 Semantic Search

- **Indexing:** On club/event create/update → generate embedding via OpenAI API → store in pgvector
- **Query:** User search text → embed → pgvector ANN search → re-rank by text match score
- **Hybrid:** Combine vector similarity + keyword (pg_trgm) score for best recall

5.4 AI Event Planning Assistant (Phase 2)

User: "Plan a weekend hiking trip for 10 people near Istanbul in October" | ▼ RAG Pipeline: 1. Embed user query 2. Retrieve relevant events/clubs from pgvector (top-20) 3. Retrieve weather/trail data via function calling (Maps API) 4. Build context window: [system prompt + retrieved docs + user query] 5. GPT-4o generates structured plan (JSON: dates, clubs, events, checklist) 6. Stream response to client | ▼ Structured Output: itinerary card with bookable events

5.5 LLM / Embedding Stack

Component	Choice	Notes
LLM (assistant)	GPT-4o (OpenAI)	Best reasoning, function calling, streaming. ~\$15/1M tokens.

Embedding model	text-embedding-3-small	1536-dim, cheap (\$0.02/1M tokens), state-of-art quality.
Vector DB (MVP)	pgvector on PostgreSQL	Zero extra infra. Good up to ~5M vectors.
Vector DB (Scale)	Qdrant (self-hosted)	10M+ vectors, HNSW index, filtering support.
LLM fallback	Claude claude-sonnet-4-6 (Anthropic)	Use when GPT-4o capacity limited or for creative tasks.
Moderation	OpenAI Moderation API	Free, fast content safety checks on all user posts.
Reranking	Cohere Rerank	Improve RAG result quality in Phase 4+.

6. 3D and Immersive Experiences

6.1 Phased Introduction

3D/AR is a differentiator — but implementing it in MVP would consume 40% of engineering effort for a feature only useful after product-market fit. Here is the realistic timeline:

Phase	Feature	Technology	Timeline
3	Photo/video galleries for trips	Standard media (no 3D)	Month 7–9
4	Interactive map for hiking routes	Mapbox GL JS + 3D terrain	Month 9–12
4	360° photo previews	Pannellum or Photo Sphere Viewer	Month 10–12
5	3D trip destination previews	React Three Fiber + GLTF models	Month 13–18
5	Hiking route 3D visualization	Three.js + elevation data APIs	Month 15–18
5+	AR experience previews	WebXR API + 8th Wall / Niantic	Month 18+
5+	VR campsite exploration	A-Frame or Unity WebGL build	Month 20+

6.2 Technology Stack for 3D

- **Web 3D:** React Three Fiber (R3F) — declarative Three.js in React. Same component model as your UI. Ideal for trip cards with 3D globe or terrain preview.
- **Mobile 3D:** Expo-three + React Native Three Fiber — same R3F code runs on mobile via a WebGL context. Minimal duplication.
- **Asset Format:** GLTF/GLB files compressed with Draco. Host on Cloudflare R2. Lazy-load on demand.
- **Map 3D Layer:** Mapbox GL JS with terrain3D enabled. Free up to 50K map loads/month. Covers hiking routes with real elevation data.
- **AR (Phase 5+):** WebXR Device API for browser AR. 8th Wall for maximum cross-device support (no app install required).

6.3 Asset Management Strategy

- Store 3D assets in R2 with long cache TTL (1 year + versioned URLs)
- LOD (Level of Detail): serve low-poly preview first, high-res on interaction
- Preload critical 3D assets in background after page load (React Suspense)
- Use KTX2 compressed textures to reduce GPU memory by 75%
- Lazy-load R3F bundle (350KB) only on pages that use 3D

7. Security and Compliance

7.1 Authentication & Authorization

- **Auth Provider:** Clerk — JWT-based sessions, magic links, Google/Apple OAuth, MFA, session revocation
- **RBAC Model:** Role hierarchy: Super Admin > Org Admin > Club Owner > Club Moderator > Member > Guest
- **JWT Validation:** FastAPI middleware validates Clerk JWT on every protected route. No session state on server.
- **Row-Level Security:** PostgreSQL RLS policies enforce data isolation (users cannot read private club data they are not members of).

7.2 RBAC Permission Matrix

Action	Guest	Member	Moderator	Club Owner	Org Admin	Super Admin
Browse public clubs	✓	✓	✓	✓	✓	✓
Register for events	—	✓	✓	✓	✓	✓
Create posts	—	✓	✓	✓	✓	✓
Manage club members	—	—	✓	✓	✓	✓
Create/edit events	—	—	—	✓	✓	✓
Approve registrations	—	—	✓	✓	✓	✓
Delete any content	—	—	—	—	✓	✓
Access admin panel	—	—	—	—	—	✓

7.3 Security Checklist

- **Rate Limiting:** Cloudflare WAF + FastAPI Slowapi middleware. 100 req/min per IP, 20 auth attempts/hour.
- **Input Validation:** Pydantic v2 models on all FastAPI endpoints. Zod on frontend forms. Never trust client input.
- **SQL Injection:** SQLAlchemy ORM with parameterized queries only. No raw string concatenation in SQL.
- **CORS:** Strict allowlist: only approved origins. FastAPI CORS middleware configured explicitly.
- **File Uploads:** Validate MIME type server-side (not just extension). Virus scan with ClamAV on R2 upload Lambda.
- **Secrets:** Never in code. Use Railway/AWS Secrets Manager. Rotate keys quarterly.
- **HTTPS:** TLS 1.3 everywhere. HSTS headers. Certificates auto-renewed via Let's Encrypt.
- **Audit Logging:** Every privileged action (role change, ban, delete) logged to append-only audit table with actor + timestamp.

7.4 GDPR Compliance

- Data export endpoint: `/api/me/export` returns all user data as JSON
- Account deletion: cascade delete or anonymize all PII within 30 days
- Cookie consent banner (client-side, Cookiebot or custom)
- Privacy policy + DPA with all sub-processors (OpenAI, Clerk, Stripe)
- Data residency: EU users on EU-region infrastructure (Railway EU / AWS eu-west-1)
- Minimal data collection: only collect what you display or process

8. DevOps and Deployment

8.1 CI/CD Pipeline (GitHub Actions)

On Pull Request: | Lint (ESLint + Ruff) | Type Check (tsc --noEmit + mypy) | Unit Tests (Vitest + pytest) | Security Scan (Trivy for Docker images) | On Merge to main: | Build Docker image → push to ECR/GHCR | Deploy to staging (Railway/ECS) | Run integration tests (Playwright E2E) | Notify Slack on success/failure | On Release Tag: | Promote staging image to production | Run DB migrations (Alembic) | Post-deploy smoke tests

8.2 Environment Management

Env	Purpose	Infra	Data
local	Development	Docker Compose (Postgres + Redis)	Seed data only
staging	QA + integration tests	Railway (mirrors prod config)	Anonymized prod snapshot
production	Live users	AWS ECS + RDS + ElastiCache	Real user data

8.3 Monitoring Stack

- **Error Tracking:** Sentry — automatic exception capture for FastAPI + Next.js + React Native
- **Application Metrics:** Prometheus + Grafana Cloud — request latency, error rates, DB pool usage
- **Log Aggregation:** Loki + Grafana — structured JSON logs, searchable, correlated with traces
- **Uptime:** BetterUptime or UptimeRobot — alert PagerDuty/Slack on downtime
- **Real User Monitoring:** PostHog Web Analytics — page load times, Core Web Vitals

8.4 Backup and Disaster Recovery

- Postgres: automated daily snapshots (Railway built-in) + WAL streaming to S3
- R2 media: cross-region replication to secondary bucket
- RPO target: 1 hour | RTO target: 4 hours
- Monthly DR drill: restore from backup to staging and verify
- Database migrations: always backward-compatible, never drop columns in same deploy

8.5 Testing Strategy

Type	Tool	Target Coverage	When Run
Unit Tests	Vitest (FE) + pytest (BE)	80%+ logic/utils	On every commit
Integration Tests	pytest + httpx (FastAPI)	All API endpoints	On PR merge
E2E Tests	Playwright	Critical user flows	Pre-production deploy
Load Tests	k6 or Locust	500 concurrent users	Weekly on staging
Security Tests	Bandit (Python) + Semgrep	All code paths	On PR

9. Product Roadmap

A phased approach that builds on validated foundations at each stage, avoiding the trap of over-building features before user demand is confirmed.

Phase 1: MVP — Build & Launch (0–3 months)

Features	Technical Milestones	Team	Complexity
• User auth + profiles • Club discovery + search • Event listings + details • Event registration + waitlist • Organizer dashboard • Event creation/editing • Push + email notifications • Basic recommendations	• NX monorepo setup • FastAPI + Next.js scaffold • Clerk auth integration • PostgreSQL schema + migrations • R2 file uploads • Railway deployment • GitHub Actions CI/CD • Meilisearch integration	• 1 Full-stack engineer • 1 Backend/ML engineer • 1 Designer (part-time)	Medium

Phase 2: Validation & Growth (3–6 months)

Features	Technical Milestones	Team	Complexity
• Mobile app (React Native) • Saved/favorite events • Organizer analytics • Approval workflows • Reviews & ratings • Payment + ticketing (Stripe) • Referral system • Multi-language support	• Expo app to App Store • Stripe integration • Read replica setup • CDN optimization • PostHog analytics • A/B testing framework • Load testing to 10K users	• +1 Mobile engineer • +1 Backend engineer • Growth marketer	Medium-High

Phase 3: Social Layer (6–10 months)

Features	Technical Milestones	Team	Complexity
• Activity feed • Club posts with media	• Feed architecture (fanout)	• +1 Backend engineer	High

• Likes and comments • Event highlights/stories • Follow clubs and users • Direct messaging (basic) • User reputation system	• Media CDN optimization • WebSocket real-time feed • Content moderation pipeline • Video processing workers • Social graph in Postgres	• +1 Frontend engineer	
--	---	--	--

Phase 4: AI Capabilities (10–14 months)

Features	Technical Milestones	Team	Complexity
• AI recommendation engine • Semantic search upgrade • AI event planning assistant • Smart user-community matching • AI content moderation • Predictive capacity alerts	• Embedding pipeline (pgvector) • RAG architecture for assistant • LightGBM ranking model • A/B test recommendation lift • Vector DB migration (Qdrant) • ML pipeline with Airflow	• +1 ML Engineer • Data Engineer	

Phase 5: Immersive Experiences (14–20 months)

Features	Technical Milestones	Team	Complexity
• Interactive 3D trip previews • Hiking route visualization • 360° venue previews • AR experience teasers • Virtual campsite tours • Mapbox 3D terrain maps	• React Three Fiber integration • GLTF asset pipeline • Mapbox GL JS 3D terrain • WebXR AR prototype • LOD asset delivery system • GPU performance optimization	• +1 3D/Graphics engineer • +1 Content team	

10. Final Recommendation

10.1 The Single Architecture I Would Choose as CTO

A TypeScript + Python Modular Monolith, deployed on Railway, growing into AWS ECS. This hits the optimal point on the complexity vs. velocity curve for a startup with 1–5 engineers.

10.2 The Stack I Would Start With Today

Layer	Technology	Why Right Now
Frontend	Next.js 15 + Tailwind + shadcn/ui	SSR + SSG + API routes in one deploy. Best-in-class DX.
Mobile	Expo (React Native) — build later (Month 3+)	Start web-only. Validate product first.
Backend	FastAPI + Python 3.12	Async, fast, Pydantic v2, native AI/ML.
Database	PostgreSQL 16 + pgvector + Redis	One DB for everything. pgvector for free embeddings.
Auth	Clerk	30-min integration. Never build auth yourself.
File Storage	Cloudflare R2	S3-compatible, zero egress fees, instant CDN.
Search	Meilisearch (self-hosted or cloud)	Instant setup, typo-tolerant, free tier.
AI	OpenAI API (text-embedding-3-small + GPT-4o)	Best quality, pay-as-you-go, no GPU infra.
Deployment	Railway (Postgres + Redis + FastAPI + Next.js)	One platform, zero DevOps overhead, \$5 start.
Monitoring	Sentry + PostHog	Error tracking + product analytics from Day 1.
CI/CD	GitHub Actions + Docker	Free, fast, reproducible builds.

10.3 The Stack You Evolve Into (3–5 Years)

- **Infra:** AWS ECS + RDS Aurora + CloudFront + ElastiCache
- **Architecture:** Modular monolith → extract notification + recommendation services
- **Search:** Elasticsearch or OpenSearch for 1M+ document corpus
- **AI:** Custom fine-tuned embedding model on your event data + Qdrant
- **Mobile:** React Native mature apps with native modules (camera, maps, AR)
- **Data:** dbt + Airflow for ML feature pipelines
- **Payments:** Stripe Connect marketplace for organizer payouts

10.4 The Major Mistakes to Avoid

Mistake	Why It Kills Startups
---------	-----------------------

Starting with microservices	10× operational overhead with 0 traffic. You spend 80% on infra, 20% on features. Monolith first, always.
Building auth from scratch	3 weeks minimum, security risks guaranteed. Use Clerk. Ship features instead.
Premature optimization	Optimizing for 1M users when you have 100. Postgres can handle 100K active users easily.
Skipping TypeScript	Without types, every refactor is a guessing game. TS pays back from week 2 onward.
No feature flags	Without feature flags, you cannot safely deploy incrementally or A/B test. Add PostHog flags early.
Building mobile before web-PMF	A React Native app before product-market fit drains engineering capacity. Web MVP first.
Over-investing in 3D/AR early	Immersive features are a differentiator at scale. They are a distraction at 0–1K users.
Ignoring observability	You will be debugging production blind. Sentry + Grafana from Day 1, not Month 6.
Manual deployments	One engineer forgetting a step breaks production. Automate CI/CD before your first real user.
Not planning for GDPR	Retrofitting GDPR at 100K users means rewriting data pipelines. Data minimization from Day 1.

Bottom Line: Ship a working web app in 3 months with 2–3 engineers. Validate that people actually want to find and join communities on your platform. Then pour investment into growth, social, and AI. The best architecture is the one that lets you learn fastest — not the most technically impressive one.